TD 1

Théorie des jeux

Exercice 1 : Stratégies au jeu de Nim . . . . . [⋆]

Nous étudions dans tout cet exercice le jeu de Nim $(n = 5, p = 2)$.

Implémentation du jeu — On commence par implémenter le graphe du jeu sous forme de dictionnaire.

1) Représenter graphiquement un graphe orienté *simple* G_s associé au jeu de Nim $(n = 5, p = 2)$. Définir formellement le graphe $G_s = (S, A)$, ainsi que l'origine s_0 et les sommets finaux F.
2) Rappeler les définitions d'une matrice d'adjacence et d'une liste d'adjacence. Donner la matrice d'adjacence et la liste d'adjacence du graphe G_s défini précédemment. Quelle est la complexité $C(n, p)$ pour implémenter la liste d'adjacence du graphe simple d'un jeu de Nim (n, p) ?
3) Représenter graphiquement un graphe orienté *biparti* G associé au jeu de Nim $(n = 5, p = 2)$. Chaque sommet est de la forme $s = (i, j)$, où $0 \leq i \leq n$ représente le nombre d'objets, et $j = 1$ ou 2 le joueur. Implémenter ce graphe en `python` sous forme d'un dictionnaire `G`, où chaque clé est un sommet `(i,j)` et chaque valeur `G[(i,j)]` représente la liste des successeurs de `(i,j)`.
4) En pratique, il est fastidieux de définir manuellement le graphe biparti associé au jeu de Nim pour les grandes valeurs de n. On propose donc de définir une fonction récursive `nim` qui construit le graphe orienté biparti associé au jeu de Nim $(n, p = 2)$. Commenter et compléter le code ci-dessous. Effectuer l'exécution pas à pas de l'appel `G={};nim(G,5,1)`. Exécuter ce code sur `python` puis vérifier que la variable `G` correspond bien au graphe orienté biparti du jeu de Nim $(n = 5, p = 2)$.

```python
def nim(G,n,j):
    """
    Entrée : G (dict), n (int > 1), j (1 ou 2)
    Sortie : None
    """
    adv = 1+j%2
    if n >= 2:
        G[(n,j)] =
        nim(G,n-1,adv)
        nim(G,n-2,adv)
    elif n == 1:
        G[(n,j)] =
        nim(G,n-1,adv)
    else:
        G[(n,j)] =
```

5) Bonus : généraliser la fonction `nim` à une valeur de p quelconque. Utiliser cette fonction pour générer le graphe représentatif du jeu de nim $(n = 9, p = 3)$.

Nombre de Grundy — On se propose de déterminer les stratégies gagnantes du jeu de Nim à partir d'un théorème mathématique appelé théorème de Sprague-Grundy. À chaque sommet (i, j) appartenant au joueur j contenant i objets, on associe le nombre noté $g(i)$, appelé nombre de Grundy, défini par :

$$\begin{cases} g(i) = i \text{ si } 1 \leq i \leq p \text{ (sommet final)}, \\ g(i) = \min\{\mathbb{N} \backslash \{g(i-\ell), \forall \ell \in [\![1, p]\!]\}\} \text{ sinon.} \end{cases}$$

L'ensemble $\{g(i-\ell), \forall \ell \in [\![1, p]\!]\}$ représente l'ensemble des nombres de Grundy des successeurs de i. On commence par étudier le jeu de Nim pour $p = 2$. Par exemple, depuis la position $i = 4$ on peut atteindre les positions 3 et 2. Si $g(3) = 0$ et $g(2) = 2$, alors $g(4) = \min\{1, 3, 4, ...\} = 1$.

6) Montrer par récurrence sur i que $g(i)$ est égal au reste de la division euclidienne de i par 3, soit

$$\forall i \in \mathbb{N}^*, g(i) = r, \text{ avec } i = 3q + r, 0 \leq r \leq 2. \tag{1}$$

On admet que ce résultat se généralise à p quelconque :

$$\forall i \in \mathbb{N}, g(i) = r, \text{ avec } i = (p+1)q + r, 0 \leq r \leq p. \tag{2}$$

7) Le théorème de Sprague-Grundy, établi dans les années 1930, affirme que tout sommet $(i, 2) \in S_2$ dont le nombre de Grundy vaut 0 est une position gagnante pour le joueur $j = 1$. En déduire une stratégie gagnante f pour le joueur $j = 1$ suivant la valeur de n. Existe-t-elle toujours ?

Algorithme min-max — On souhaite appliquer l'algorithme min-max au jeu de Nim.

8) Représenter l'arbre de décision associé au graphe orienté *biparti* G du jeu de Nim $(n = 5, p = 2)$.

9) Appliquer à la main l'algorithme min-max sur ce graphe. En déduire s'il existe une stratégie gagnante pour le joueur $j = 1$. Proposer une stratégie gagnante.

10) Pour implémenter l'algorithme min-max, nous souhaitons parcourir le graphe. Rappeler le principe des deux types de parcours de graphe (largeur et profondeur) et la structure de données utilisée dans chaque cas (pile ou file). À quel type de parcours correspond le code suivant ?

```python
def parcours(G,s0):
    """
    Entrée : graphe G (dictionnaire), sommet initial s0 = (j,i)
    Sortie : None
    """
    atraiter = [s0]
    while len(atraiter) > 0:
        s = atraiter.pop()
        for v in G[s]:
            atraiter.append(v)
```

11) Pourquoi l'algorithme itératif ci-dessus n'est-t-il pas adapté pour appliquer l'algorithme min-max ? On propose à la place de définir une fonction récursive `minmax`. Commenter et compléter le code ci-dessous. Effectuer l'exécution pas à pas de l'appel `minmax(G,(1,5))` sur le dictionnaire `G` défini en Sec. 1 et comparer aux résultats des questions précédentes.

```python
def minmax(G,s):
    """
    Entrée : graphe G (dictionnaire), sommet initial s
    Sortie : heuristique h(s) (entier)
    """
    if G[s] == []:
        if s[0] == 1:
            return ...
        else:
            return ...
    else:
        h = [minmax(G,v) for v in G[s]]
        if s[0] == 1:
            return ...
        else:
            return ...
```

Attracteur — On souhaite calculer l'attracteur du graphe orienté biparti G du jeu de nim $(n = 5, p = 2)$.

12) Rappeler la définition de l'attracteur Que vaut l'ensemble F_1 des sommets finaux pour je joueur $j = 1$? Appliquer l'algorithme de calcul de l'attracteur à la main et en déduire l'attracteur $\mathscr{A}$.

13) On souhaite implémenter le calcul de l'attracteur sur python. Pour cela, on peut se demander comment déterminer l'ensemble des sommets finaux d'un graphe G pour un joueur j. Préciser les spécifications de la fonction `final` et compléter le code suivant. Determiner sur `python` la liste `F1` des sommets finaux pour le joueur $j = 1$.

```python
def final(G,j):
    F = []          # initialisation de F
    for s in G:     # boucle sur les sommets s
        ...         # à compléter
    return F        # renvoit sommets finaux F
```

14) On donne ci-dessous une fonction définie en `python` pour calculer l'attracteur d'une graphe G défini sous forme de dictionnaire. On suppose avoir prédéfini les fonctions `existe` et `pourtout` telles que `existe(G,F,j)`$= \{s \in S_j \backslash F, \exists$ arc (s,t) avec $t \in F\}$ et `pourtout(G,F,j)`$= \{s \in S_j \backslash F, \forall$ arc $(s,t), t \in F\}$, pour tout joueur $j = 1, 2$. Effectuer une exécution pas à pas de la commande `attract(G,F1)` où G désigne le graphe du jeu pour $n = 5$ et $p = 2$. Que vaut l'attracteur $\mathscr{A}$? Comparer aux stratégies gagnantes obtenues par les autres méthodes.

```python
def attract(G,F):
    """
    Entrée : graphe du jeu G (dict), sommets finaux F (list), joueur J (int)
    Sortie : attracteur A (list)
    """
    E = existe(G,F,1)   # de type list
    P = pourtout(G,F,2) # de type list
    A = F + E + P
    if E == [] and P == []:
        return A
    else:
        return attract(G,A)
```